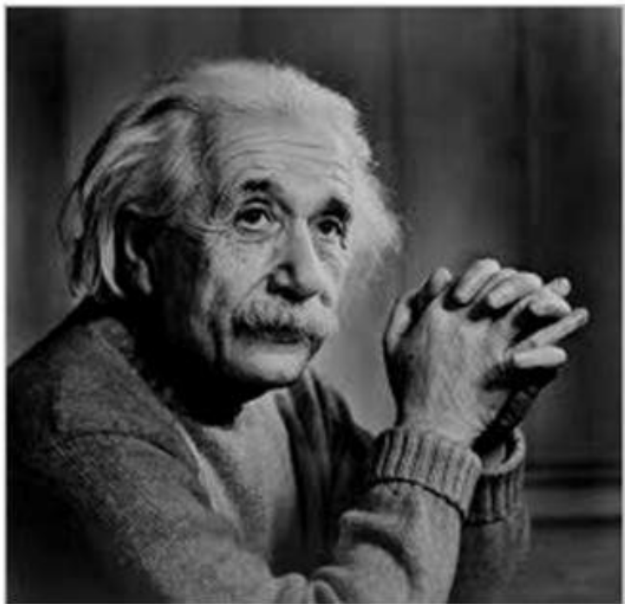


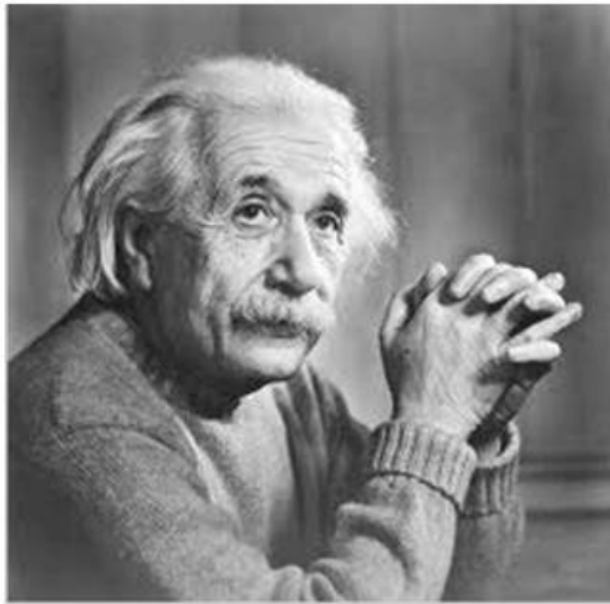
Image Processing & Image Filtering

Basic Image Processing

I



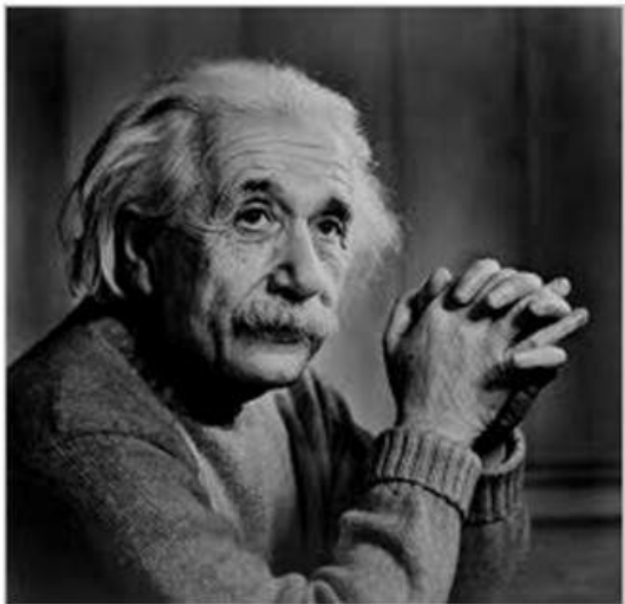
αI



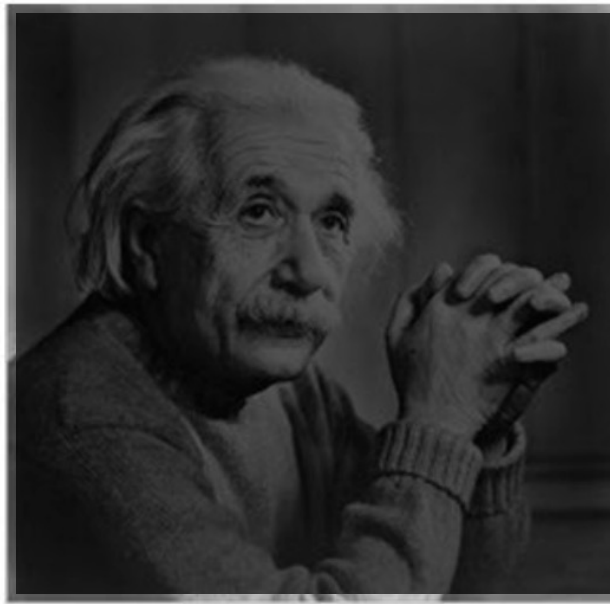
$\alpha > 1$

Basic Image Processing

I

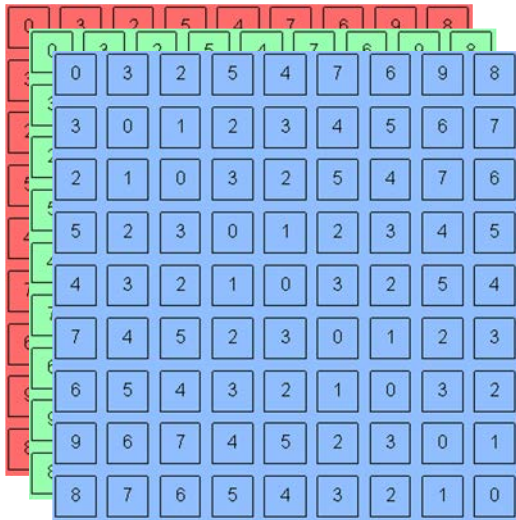


αI



$$0 < \alpha < 1$$

Color Images as Tensors



channel x height x width

Channels are usually RGB: Red, Green, and Blue

Other color spaces: HSV, HSL, LUV, XYZ, Lab, CMYK, etc

Image filtering



Image filtering

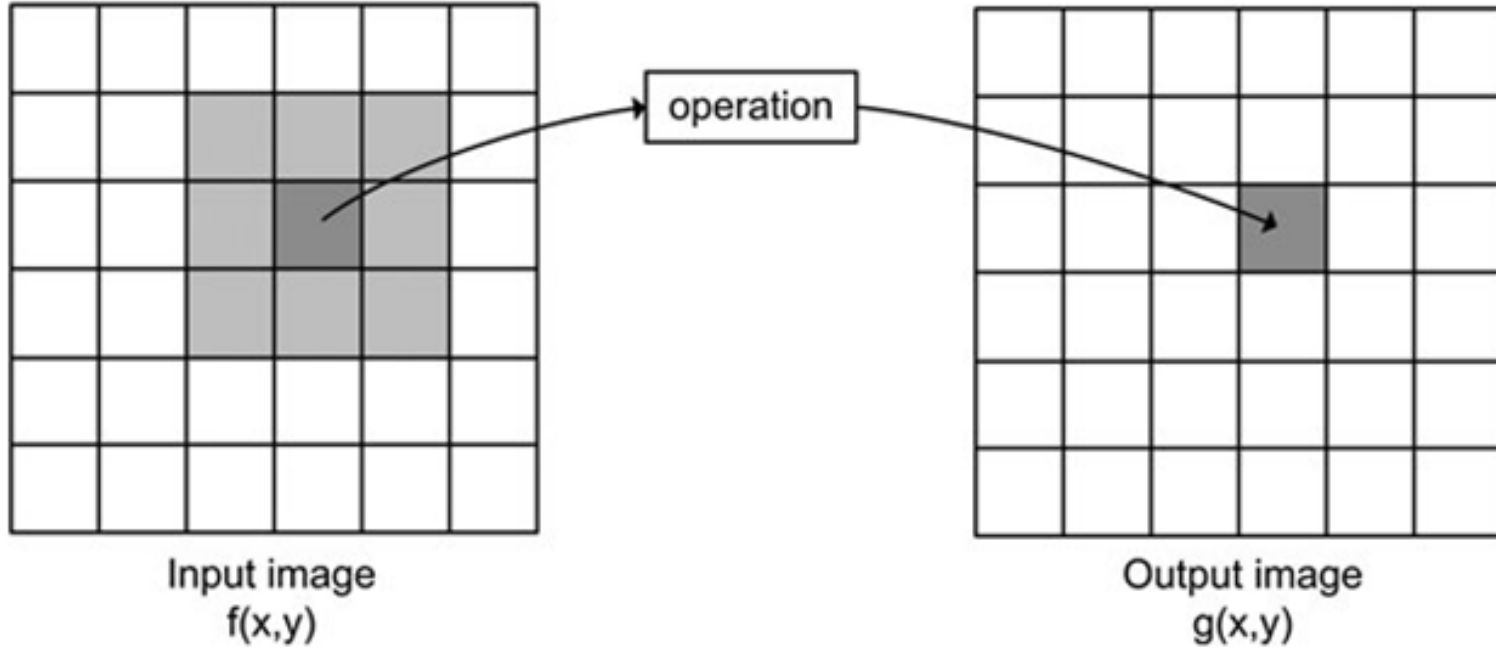
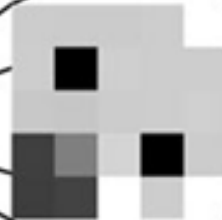
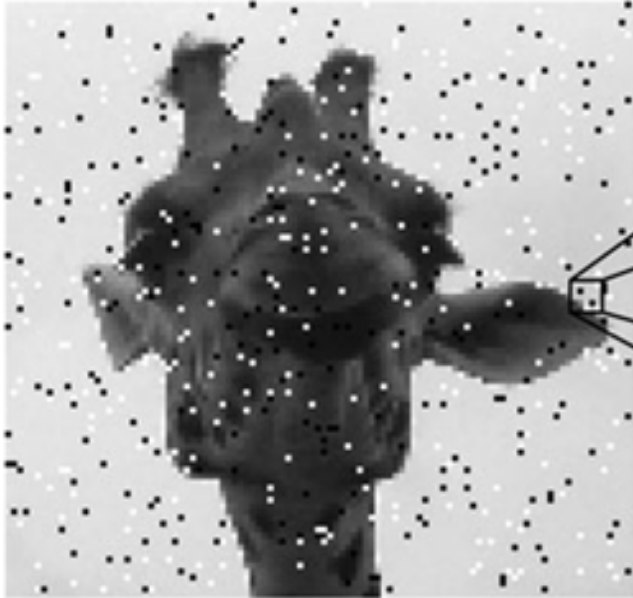


Image filtering: e.g. Mean Filter



205	204	204	206	255
206	0	208	206	206
201	199	205	206	209
61	128	213	0	205
59	65	255	206	255

Image filtering: e.g. Mean Filter

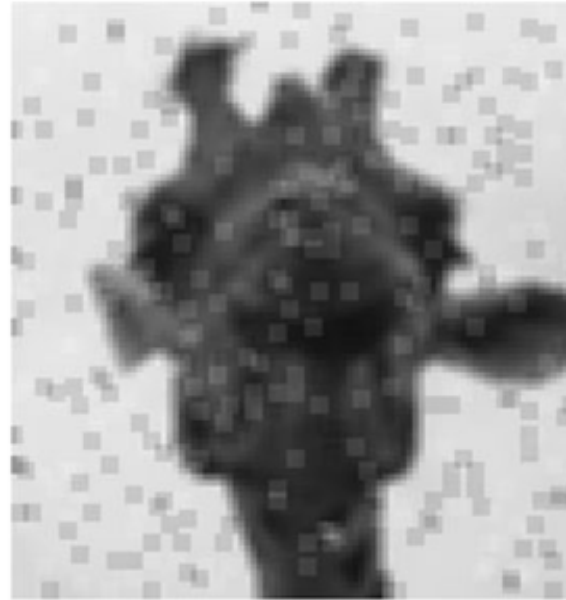
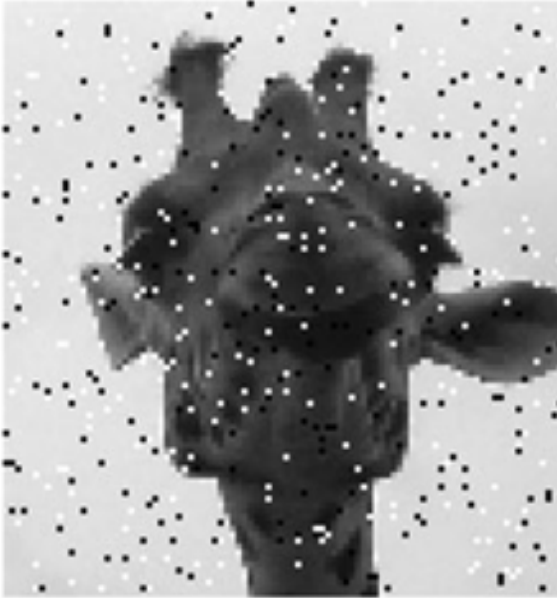


Image filtering: e.g. Median Filter

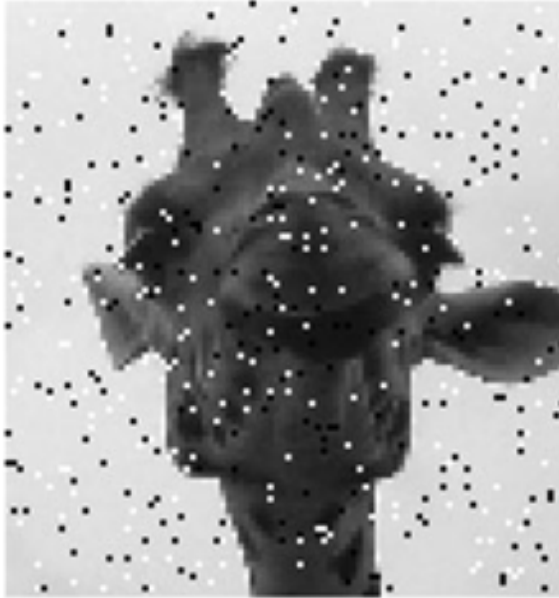
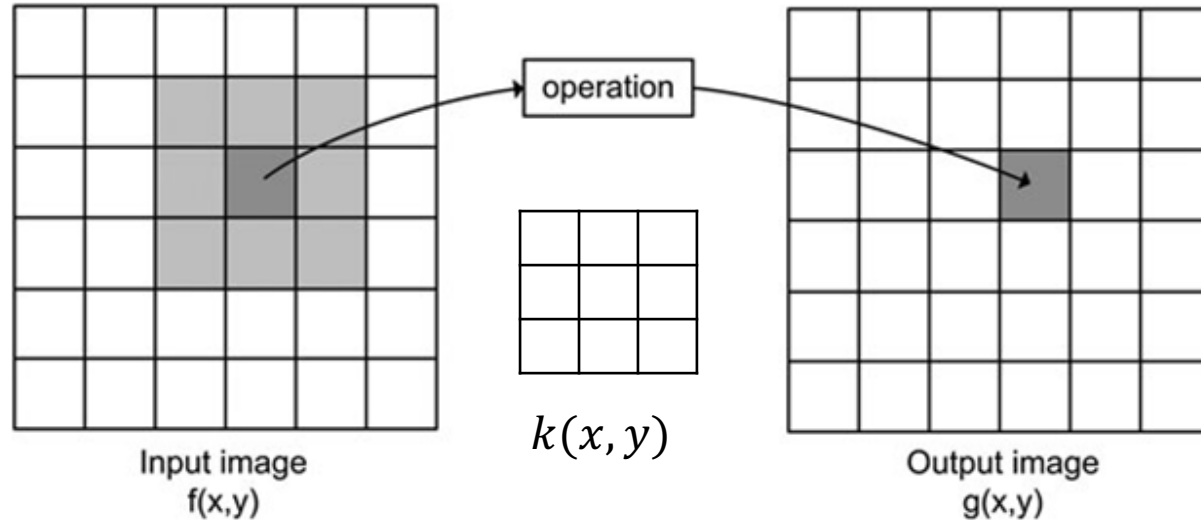


Image filtering: Convolution operator



$$g(x, y) = \sum_v \sum_u k(u, v) f(x - u, y - v)$$

(filter, kernel)

Input image

*

Weights



Output image

4	5	7	6	6
3	2	8	0	7
6	7	7	1	5
3	0	1	1	1
4	3	2	1	7

*

0	0	0
1	0	1
0	0	0

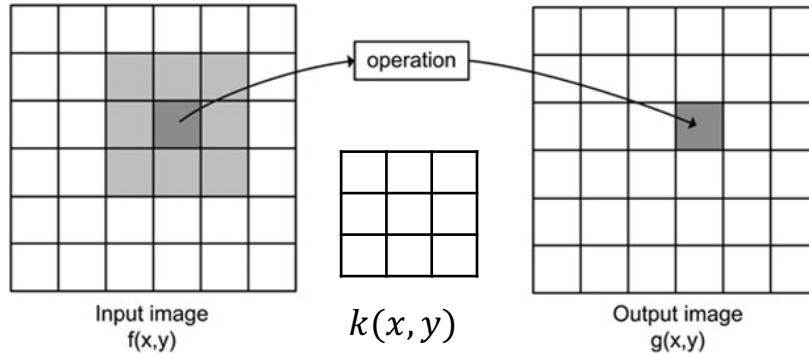


	11	2	15	
	13	8	12	
	4			

<http://www.cs.virginia.edu/~vicente/recognition/animation.gif>

Image filtering: Convolution operator

e.g. mean filter

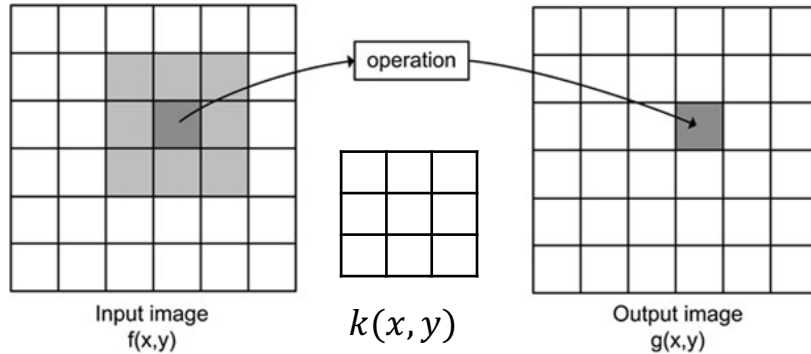


$$k(x,y) =$$

$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$

Image filtering: Convolution operator

e.g. mean filter



$$k(x,y) =$$

$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$
$1/9$	$1/9$	$1/9$

aka. box filter

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10							

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20						

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30					

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] \frac{1}{9}$$

1	1	1
1	1	1
1	1	1

$f[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[\cdot, \cdot]$

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image filtering

$$g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$f[\cdot, \cdot]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[\cdot, \cdot]$$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Box Filter

What does it do?

- Replaces each pixel with an average of its neighborhood
- Achieve smoothing effect (remove sharp features)

$$\frac{1}{9} g[\cdot, \cdot]$$

1	1	1
1	1	1
1	1	1

Image filtering: e.g. Mean Filter

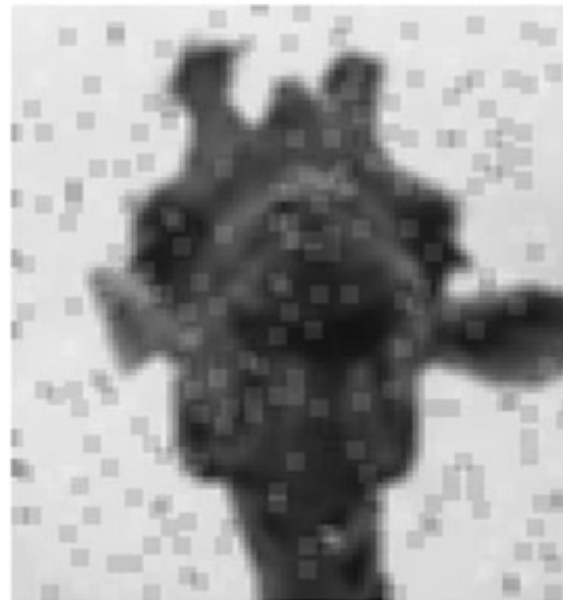
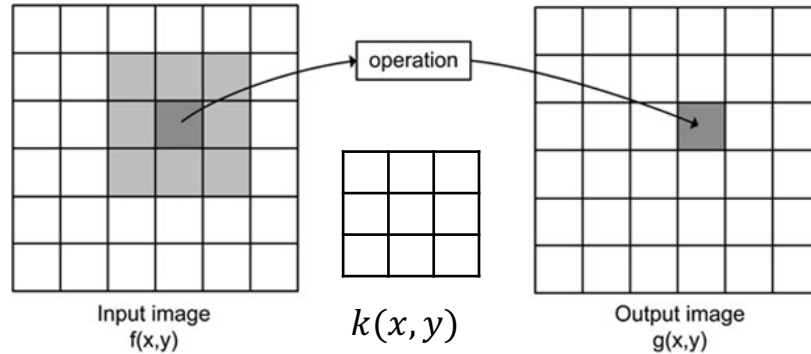
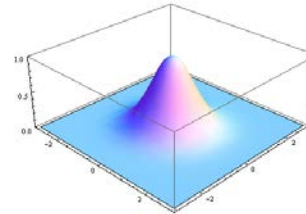


Image filtering: Convolution operator

Important filter: gaussian filter (gaussian blur)



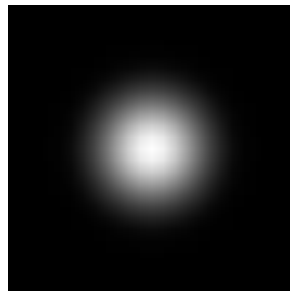
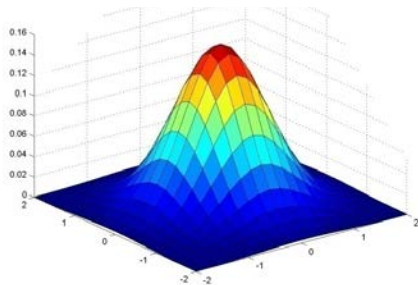
$$k(x, y) =$$



1/16	1/8	1/16
1/8	1/4	1/8
1/16	1/8	1/16

Important filter: Gaussian

- Weight contributions of neighboring pixels by nearness



0.003	0.013	0.022	0.013	0.003
0.013	0.059	0.097	0.059	0.013
0.022	0.097	0.159	0.097	0.022
0.013	0.059	0.097	0.059	0.013
0.003	0.013	0.022	0.013	0.003

5 x 5, $\sigma = 1$

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

Image filtering: Convolution operator e.g. gaussian filter (gaussian blur)



Practical matters

- What about near the edge?
 - the filter window falls off the edge of the image
 - need to extrapolate
 - methods:
 - clip filter (black)
 - wrap around
 - copy edge
 - reflect across edge



Practice with linear filters



Original

0	0	0
0	1	0
0	0	0

?

Practice with linear filters



Original

0	0	0
0	1	0
0	0	0



Filtered
(no change)

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0

?

Practice with linear filters



Original

0	0	0
0	0	1
0	0	0



Shifted left
By 1 pixel

Practice with linear filters



Original

0	0	0
0	2	0
0	0	0

−

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

?

(Note that filter sums to 1)

Practice with linear filters



Original

0	0	0
0	2	0
0	0	0

-

$\frac{1}{9}$

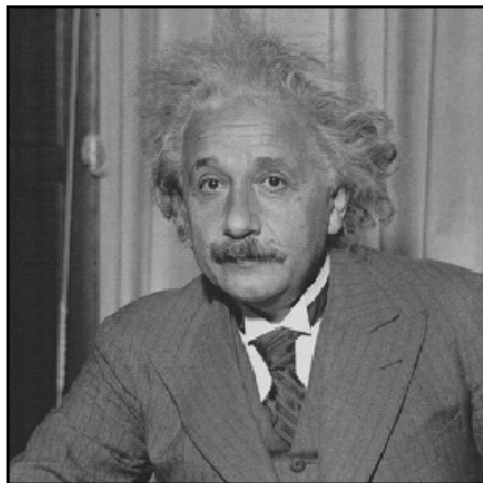
1	1	1
1	1	1
1	1	1



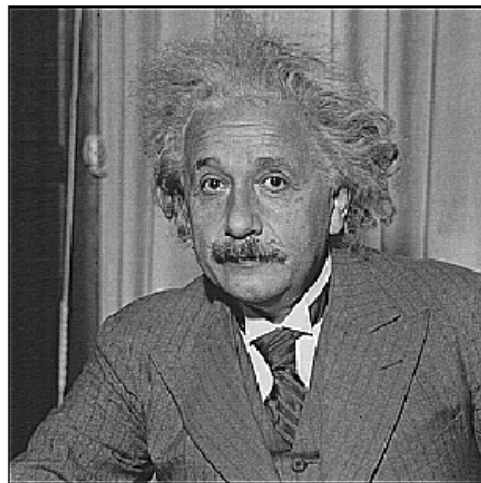
Sharpening filter

- Accentuates differences with local average

Sharpening



before



after

Key properties of linear filters

Linearity:

$$\text{imfilter}(I, f_1 + f_2) = \\ \text{imfilter}(I, f_1) + \text{imfilter}(I, f_2)$$

Shift invariance: same behavior regardless of pixel location

$$\text{imfilter}(I, \text{shift}(f)) = \text{shift}(\text{imfilter}(I, f))$$

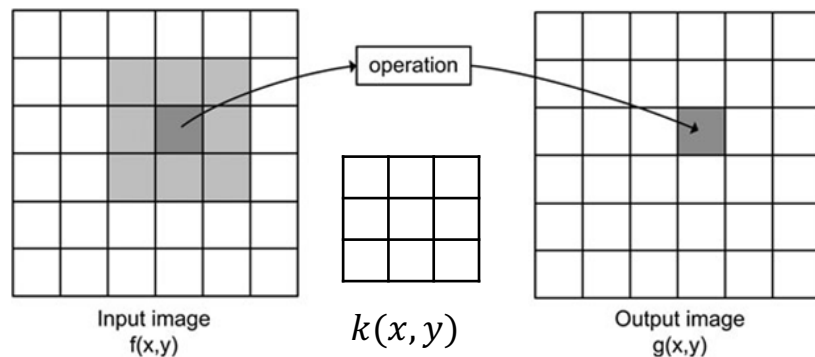
Any linear, shift-invariant operator can be represented as a convolution

Image filtering: Convolution operator

- Enhance images
 - Denoise, resize, increase contrast, etc.
- Extract information from images
 - Texture, edges, distinctive points, etc.
- Detect patterns
 - Template matching
- Deep Convolutional Networks

Image filtering: Convolution operator

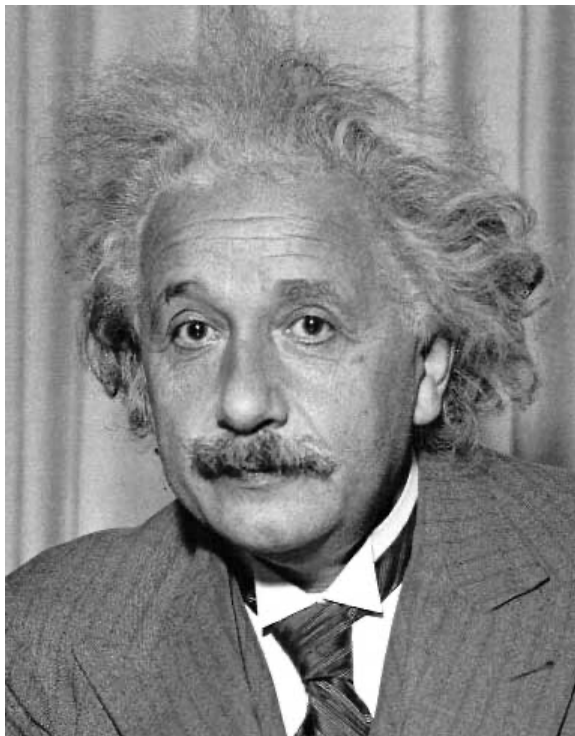
Important Filter: Sobel operator



$$k(x,y) =$$

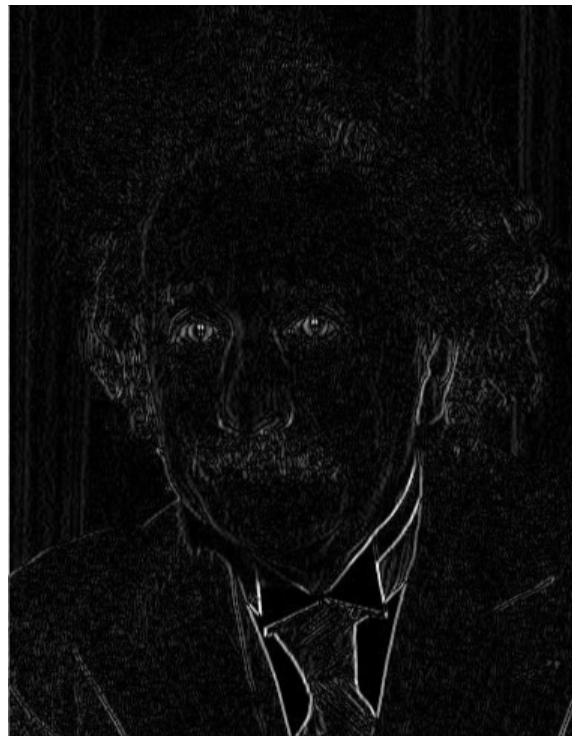
1	0	-1
2	0	-2
1	0	-1

Other filters



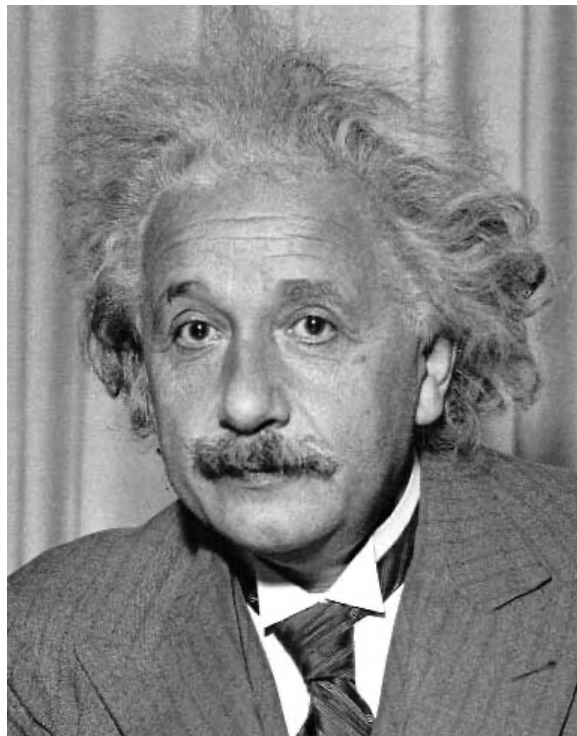
1	0	-1
2	0	-2
1	0	-1

Sobel



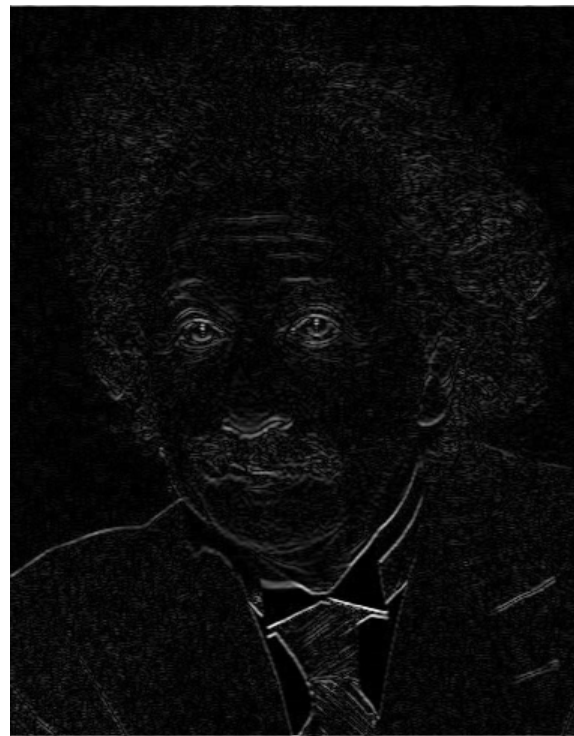
Vertical Edge
(absolute value)

Other filters



1	2	1
0	0	0
-1	-2	-1

Sobel

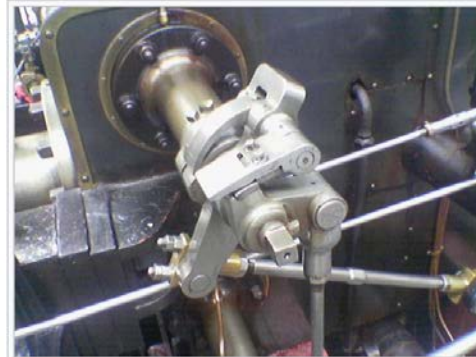


Horizontal Edge
(absolute value)

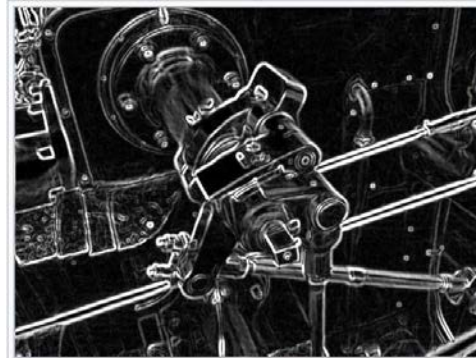
Sobel operators are equivalent to 2D partial derivatives of the image

- Vertical sobel operator – Partial derivative in X (width)
- Horizontal sobel operator – Partial derivative in Y (height)
- Can compute magnitude and phase at each location.
- Useful for detecting edges

https://en.wikipedia.org/wiki/Sobel_operator



A color picture of a steam engine



The Sobel operator applied to that image



Sobel filters are (approximate) partial derivatives of the image

Let $f(x, y)$ be your input image, then the partial derivative is:

$$\frac{\partial f(x, y)}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x + h, y) - f(x, y)}{h}$$

Also:

$$\frac{\partial f(x, y)}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x + h, y) - f(x - h, y)}{2h}$$

But digital images are not continuous, they are discrete

Let $f[x, y]$ be your input image, then the partial derivative is:

$$\Delta_x f[x, y] = f[x + 1, y] - f[x, y]$$

Also:
$$\Delta_x f[x, y] = f[x + 1, y] - f[x - 1, y]$$

But digital images are not continuous, they are discrete

Let $f[x, y]$ be your input image, then the partial derivative is:

$$\Delta_x f[x, y] = f[x + 1, y] - f[x, y]$$

$$k(x, y) =$$

-1	1
----	---

Also: $\Delta_x f[x, y] = f[x + 1, y] - f[x - 1, y]$

$$k(x, y) =$$

-1	0	1
----	---	---

Sobel Operators Smooth in Y and then Differentiate in X

$$k(x, y) = \begin{array}{|c|} \hline 1 \\ \hline 2 \\ \hline 1 \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 0 & -1 \\ \hline 2 & 0 & -2 \\ \hline 1 & 0 & -1 \\ \hline \end{array}$$

Similarly to differentiate in Y

For a detailed analysis of why it is a good idea to smooth in other direction first, see Chapter 10 in the book on Digital Image Processing by Gonzalez & Woods

http://web.ipac.caltech.edu/staff/fmasci/home/astro_refs/Digital_Image_Processing_2ndEd.pdf

Variations

Roberts

0	-1
1	0

-1	0
0	1

Prewitt

1	0	-1
1	0	-1
1	0	-1

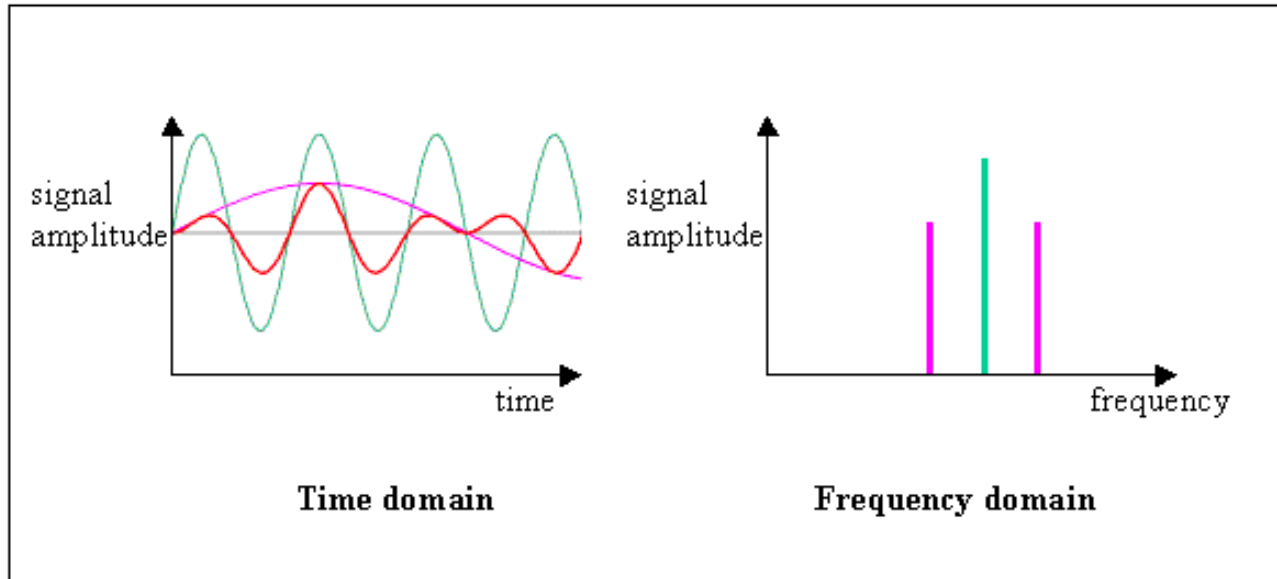
-1	-1	-1
0	0	0
1	1	1

Frei-Chen

1	0	-1
$\sqrt{2}$	0	$-\sqrt{2}$
1	0	-1

-1	$-\sqrt{2}$	-1
0	0	0
1	$\sqrt{2}$	1

Frequency



Any function can be approximated by a polynomial function

Taylor Series expansion

$$f(x) = f(a) + \frac{f'(a)}{1!} (x - a) + \frac{f''(a)}{2!} (x - a)^2 + \frac{f^{(3)}(a)}{3!} (x - a)^3 + \dots$$

...if you let your polynomial have a high degree

...AND you can compute the derivatives of the original function easily.

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

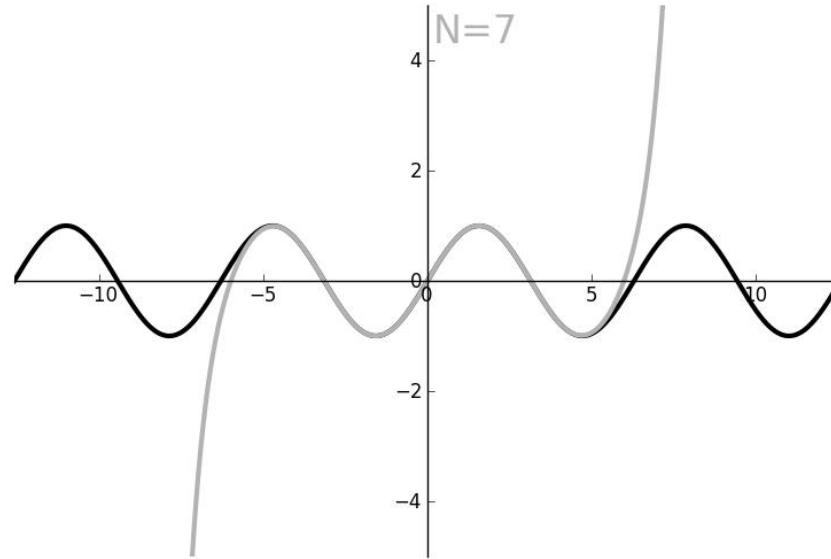
$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots \quad \text{for } |x| < 1$$

$$\tan^{-1}(x) = x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \dots \quad \text{for } |x| < 1$$

Difficult in practice



<https://brilliant.org/wiki/taylor-series-approximation/>

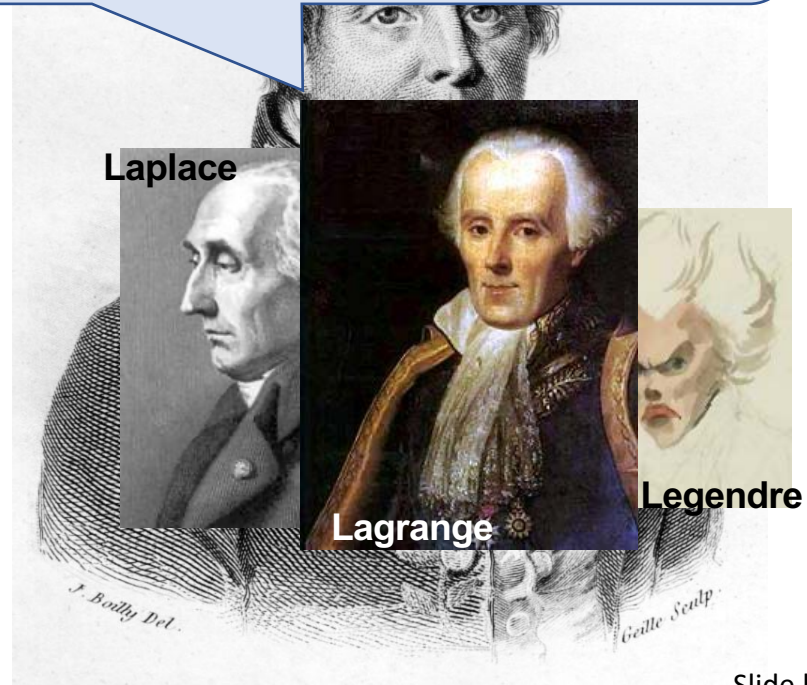
Jean Baptiste Joseph Fourier (1768-1830)

had crazy idea (1807):

Any univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies.

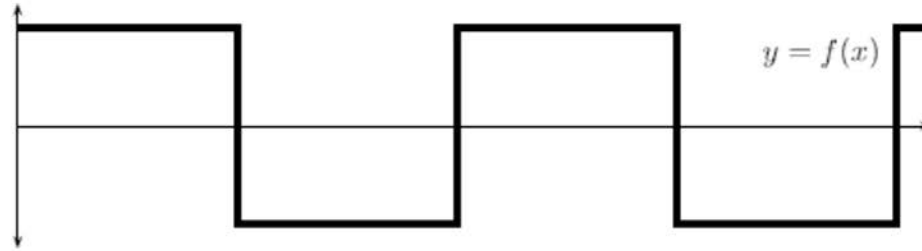
...the manner in which the author arrives at these equations is not exempt of difficulties and...his analysis to integrate them still leaves something to be desired on the score of generality and even rigour.

- Don't believe it?
 - Neither did Lagrange, Laplace, Poisson and other big wigs
 - Not translated into English until 1878!
- But it's (mostly) true!
 - called Fourier Series
 - there are some subtle restrictions

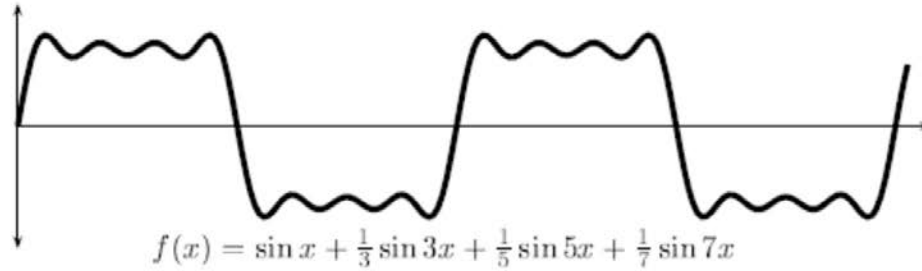


Example

Square wave



**Approximation
Using sines**



Discrete Fourier Transform

$$F(u) = \sum_{x=0}^{N-1} f(x) \left[\cos \left[-2\pi \left(\frac{xu}{N} \right) \right] + i \sin \left[-2\pi \left(\frac{xu}{N} \right) \right] \right]$$

Keep in mind Euler's Equation

$$e^{ix} = \cos x + i \sin x$$

We can compute the real and the imaginary part of the complex number.

Discrete Fourier Transform

$$F(u) = \sum_{x=0}^{N-1} f(x) \left[\cos \left[-2\pi \left(\frac{xu}{N} \right) \right] + i \sin \left[-2\pi \left(\frac{xu}{N} \right) \right] \right]$$



$$F(u) = \sum_{x=0}^{N-1} f(x) \exp \left[-2\pi i \left(\frac{xu}{N} \right) \right]$$

Discrete Fourier Transform

$$F(u) = \sum_{x=0}^{N-1} f(x) \exp \left[-2\pi i \left(\frac{xu}{N} \right) \right]$$

Inverse Discrete Fourier Transform

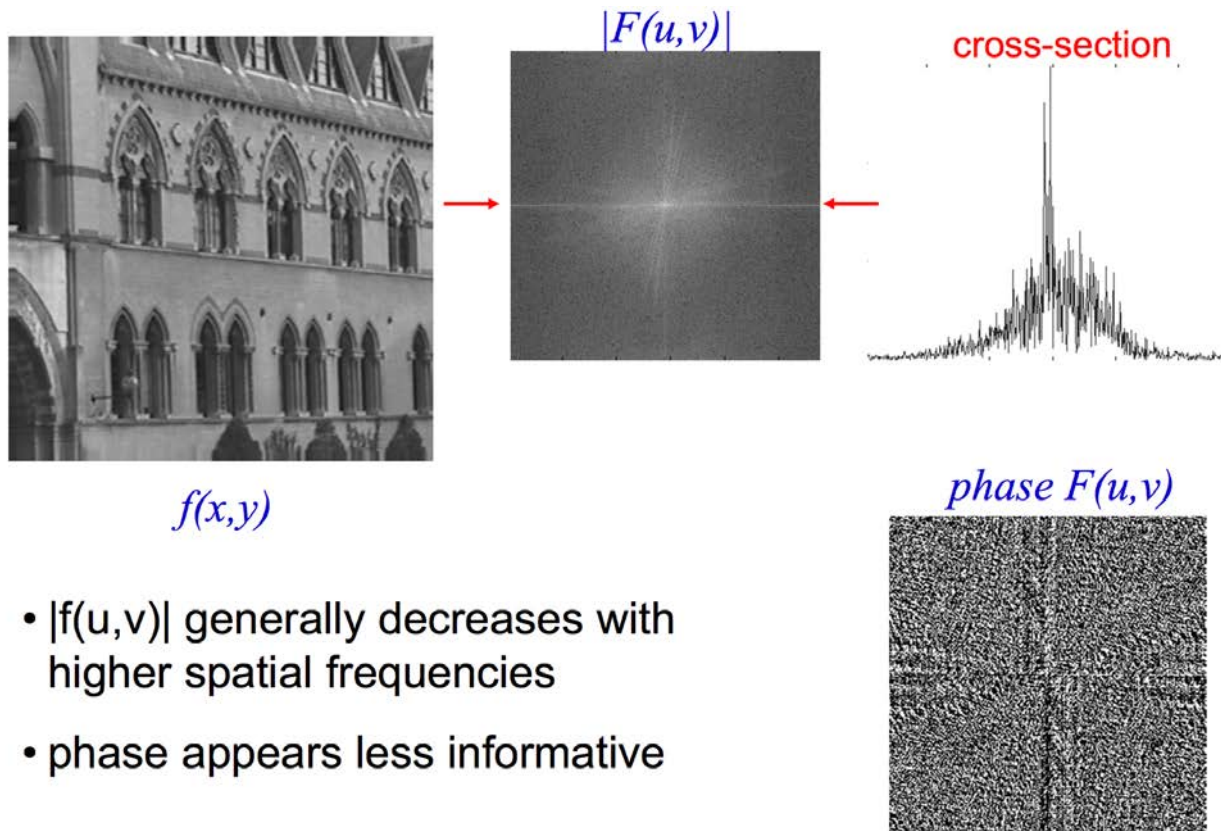
$$f(x) = \frac{1}{N} \sum_{u=0}^{N-1} F(u) \exp \left[2\pi i \left(\frac{xu}{N} \right) \right]$$

More generally for images (2D DFT and iDFT)

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \exp \left[-2\pi i \left(\frac{xu}{M} + \frac{yv}{N} \right) \right]$$

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) \exp \left[2\pi i \left(\frac{xu}{M} + \frac{yv}{N} \right) \right]$$

Discrete Fourier Transform - Visualization



- $|f(u,v)|$ generally decreases with higher spatial frequencies
- phase appears less informative

Fourier Transform

- Fourier transform stores the magnitude and phase at each frequency
 - Magnitude encodes how much signal there is at a particular frequency
 - Phase encodes spatial information (indirectly)
 - For mathematical convenience, this is often notated in terms of real and complex numbers

Amplitude: $A = \pm \sqrt{R(\omega)^2 + I(\omega)^2}$

Phase: $\phi = \tan^{-1} \frac{I(\omega)}{R(\omega)}$

Image Filtering in the Frequency Domain

original

$f(x,y)$



$|F(u,v)|$

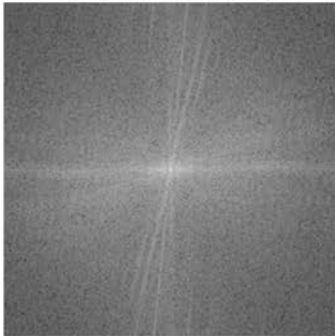


Image Filtering in the Frequency Domain

original

low pass

$f(x,y)$



$|F(u,v)|$

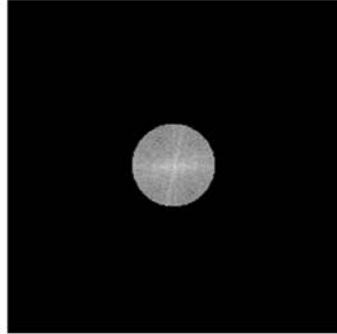
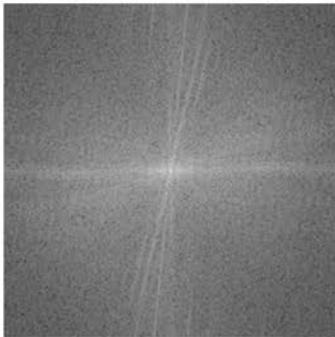
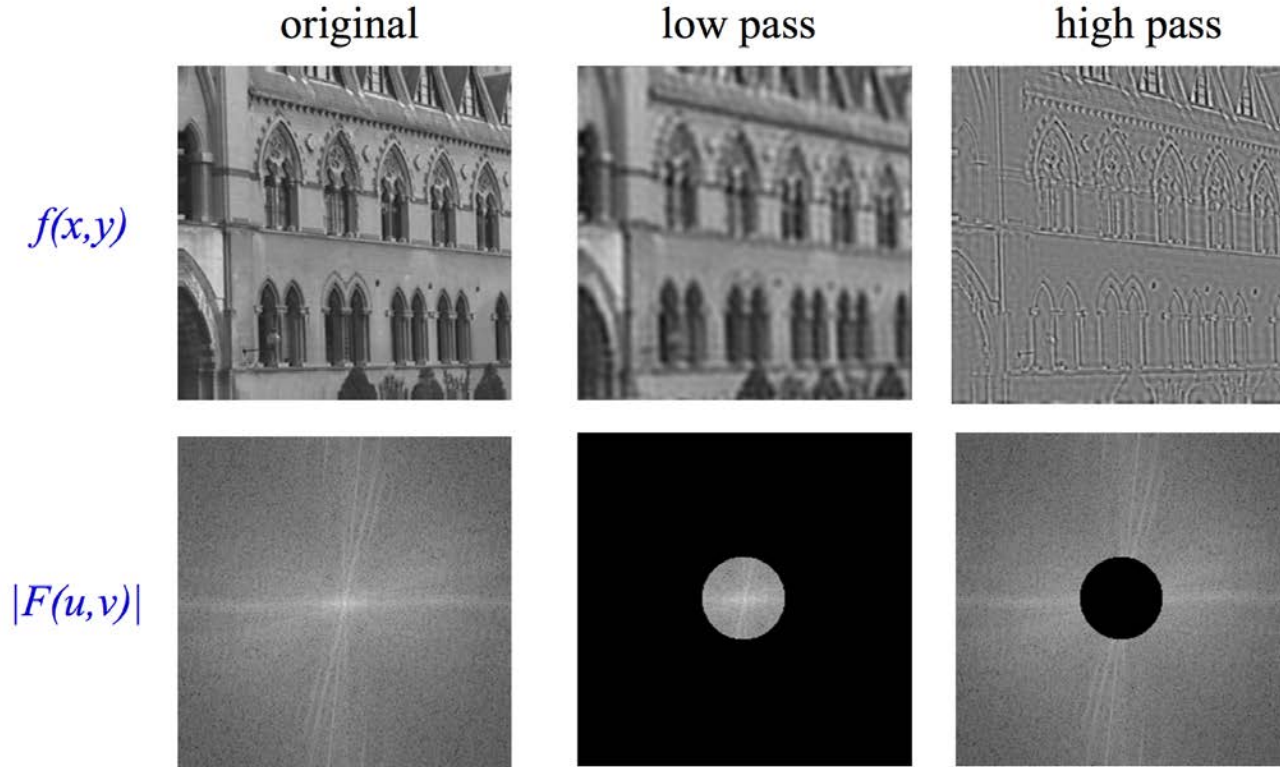


Image Filtering in the Frequency Domain



The Convolution Theorem

- The Fourier transform of the convolution of two functions is the product of their Fourier transforms

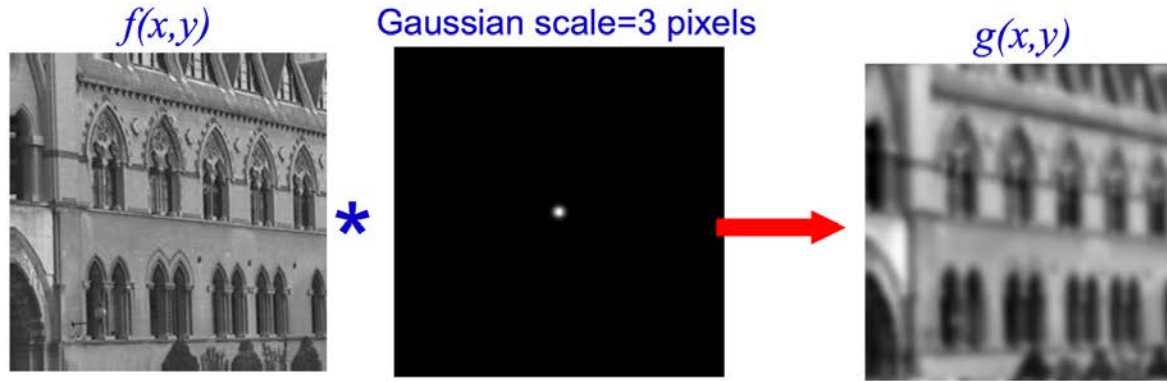
$$F[g * h] = F[g]F[h]$$

- **Convolution** in spatial domain is equivalent to **multiplication** in frequency domain!

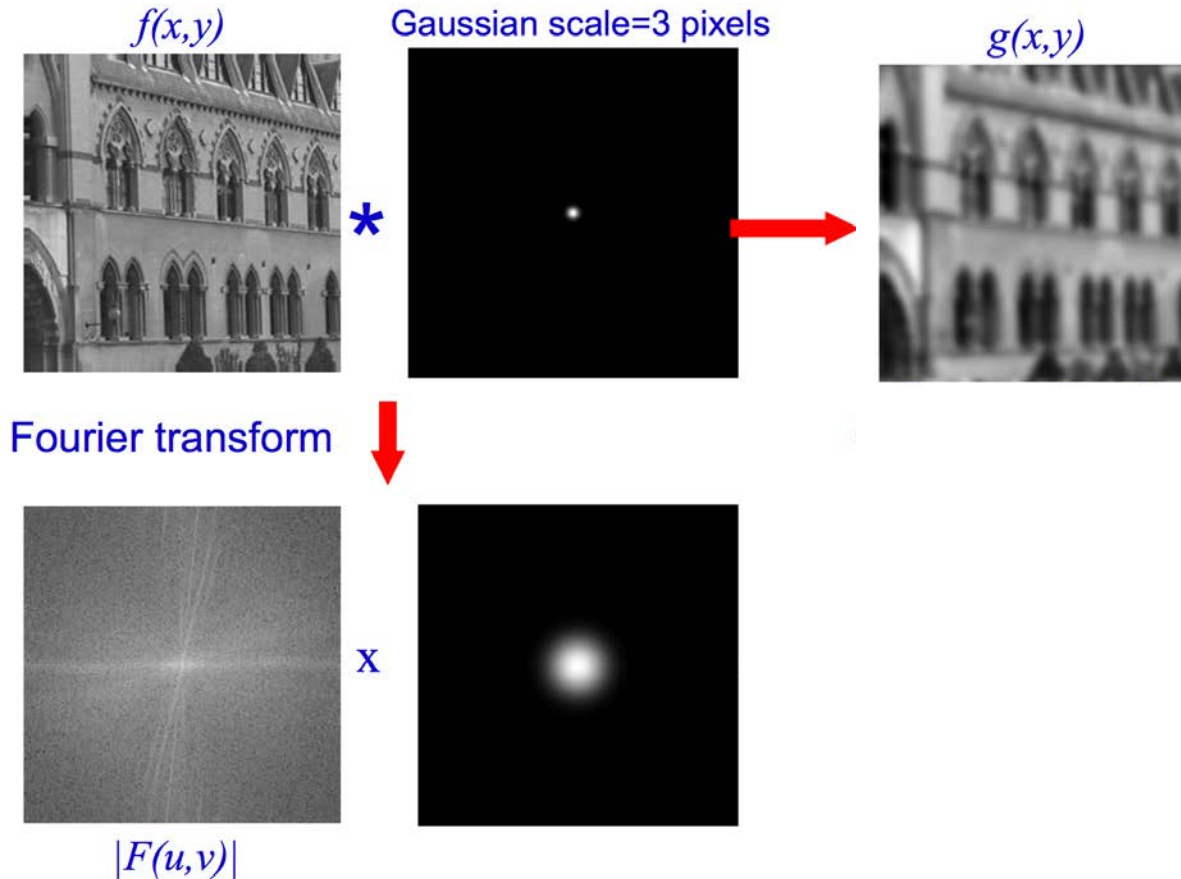
$$g * h = F^{-1}[F[g]F[h]]$$

How can this be useful?

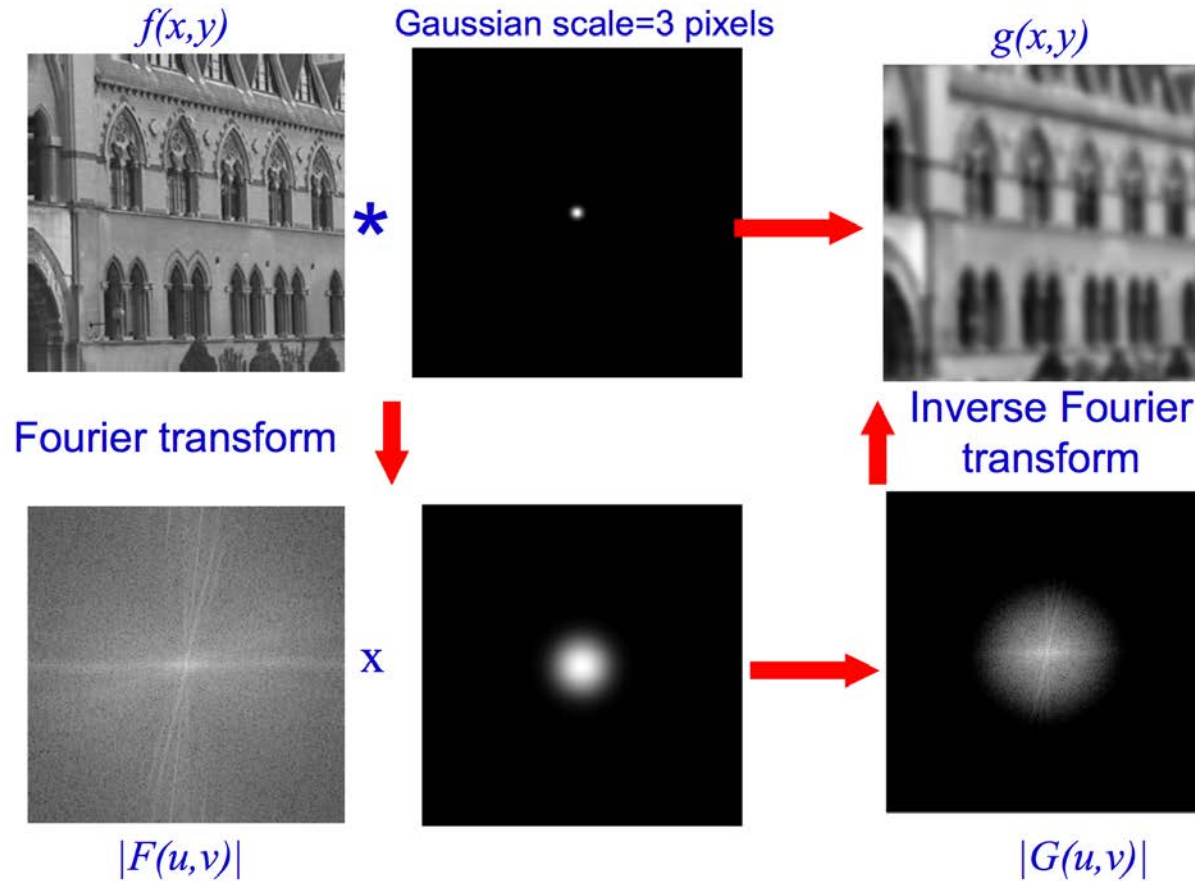
Blurring in the Time vs Frequency Domain



Blurring in the Time vs Frequency Domain



Blurring in the Time vs Frequency Domain



Why Frequency domain?

- Because the Discrete Fourier Transform can be computed fast using the Fast Fourier Transform FFT algorithm.
- Because the running time does not depend on the size of the kernel matrix.
- However rarely used these days because most filters used in Computer vision are 3×3 , 5×5 , e.g. relatively small.

Final Thoughts – JPEG Image Compression

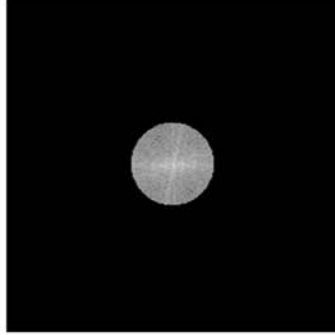
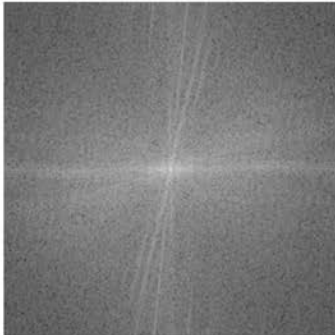
original

low pass

$f(x,y)$



$|F(u,v)|$



- Small amount of information can recover almost the original image with some loss in resolution.
- Images are dominated by low frequency information. e.g. no need to store repeated pixels.
- In practice JPEG uses a simpler transformation called Discrete Cosine Transform DCT.